

DS UNIT-5

1. Differentiate between supervised and unsupervised learning?

A.

Aspect	Supervised Learning	Unsupervised Learning
Data Type	Labeled data (input-output pairs)	Unlabeled data (only inputs)
Goal	Predict outcomes or map inputs to known outputs	Discover hidden patterns or structure
Common Tasks	Classification, Regression	Clustering, Dimensionality Reduction
Algorithms	Linear Regression, Decision Trees, SVM, k-NN	K-means, Hierarchical Clustering, PCA
Model Training	Learns from known labels	Learns from data structure without labels
Evaluation Metrics	Accuracy, Precision, Recall, RMSE	Silhouette Score, Inertia, Variance Explained
Example Use Cases	Email spam detection, loan default prediction	Customer segmentation, anomaly detection
Output	Predicted labels or values	Groupings, clusters, or reduced feature sets

2. Write any two commonly used distance measures in clustering?

A. 1. Euclidean Distance

- **Definition:** The straight-line distance between two points in Euclidean space.
- **Formula** (for two points $x = (x_1, x_2, \dots, x_n)$ and $y = (y_1, y_2, \dots, y_n)$):
$$d(x, y) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \dots + (x_n - y_n)^2}$$
- **Use Case:** Common in K-means clustering when features are continuous and scaled.

2. Manhattan Distance (L1 Norm)

- **Definition:** The sum of absolute differences between coordinates.
- **Formula:**
$$d(x, y) = |x_1 - y_1| + |x_2 - y_2| + \dots + |x_n - y_n|$$
- **Use Case:** Useful when dealing with high-dimensional data or when movement is restricted to grid-like paths (e.g., city blocks).

3. What is K-means clustering.

A. Step 1: What Is K-means Clustering?

K-means is an **unsupervised learning algorithm** used to **partition data into K distinct clusters** based on similarity. Each cluster is defined by its **centroid** (mean of points in the cluster).

Step 2: How It Works

1. **Choose K:** Decide the number of clusters.
2. **Initialize Centroids:** Randomly select K points as initial centroids.
3. **Assign Points:** Each data point is assigned to the nearest centroid (based on Euclidean distance).
4. **Update Centroids:** Recalculate centroids as the mean of all points in each cluster.
5. **Repeat:** Steps 3 and 4 until centroids no longer change significantly (convergence).

Step 3: Example

Suppose you have customer data with features like age and income:

- You choose $K = 3$ to segment customers into three groups.
- K-means will group customers such that those in the same cluster have similar age and income.
- Useful for **market segmentation**, **image compression**, or **anomaly detection**.

4. Name two metrics used to evaluate clustering quality.

A. 1. Silhouette Score

- Measures how similar a data point is to its own cluster compared to other clusters.
- Ranges from **-1 to +1**:
 - Close to **+1**: well-clustered
 - Close to **0**: overlapping clusters
 - Negative: misclassified
- Useful for assessing both **cohesion** and **separation**.

2. Davies–Bouldin Index

- Evaluates the average similarity between each cluster and its most similar one.
- Lower values indicate **better clustering** (less overlap and tighter clusters).
- Based on **intra-cluster distance** and **inter-cluster separation**.

5. Explain the Role of Similarity and Distance in Clustering.

A. Step 1: What Is Clustering

Clustering is an **unsupervised learning technique** that groups data points based on how similar they are. The goal is to ensure that:

- Points **within the same cluster** are **similar** to each other.
- Points in **different clusters** are **dissimilar**.

Step 2: Role of Distance

Distance measures how **far apart** two data points are. Common distance metrics include:

- **Euclidean distance:** Straight-line distance
- **Manhattan distance:** Sum of absolute differences
- **Cosine distance:** Based on angle between vectors

In clustering algorithms like **K-means**, distance determines:

- Which cluster a point belongs to (nearest centroid)
- How centroids are updated
- How compact and well-separated clusters are

Step 3: Role of Similarity

Similarity is the **inverse of distance**—it measures how **alike** two points are. Higher similarity means lower distance.

- In **hierarchical clustering**, similarity scores guide which clusters to merge.
- In **density-based clustering** (e.g., DBSCAN), points are grouped if they are close enough (high similarity).

Step 4: Example

Suppose you're clustering customers based on age and income:

- Two customers with similar age and income will have **low Euclidean distance** → high similarity → likely in the same cluster.
- A customer with very different income will have **high distance** → low similarity → placed in a different cluster.

6. Explain the properties of a good clustering with examples.

A. 1. High Intra-Cluster Similarity

- **Definition:** Data points within the same cluster should be very similar to each other.
- **Example:** In customer segmentation, people in the same cluster might have similar spending habits and age.
- **Why it matters:** Ensures that clusters are meaningful and coherent.

2. Low Inter-Cluster Similarity

- **Definition:** Data points in different clusters should be dissimilar.
- **Example:** One cluster of customers may prefer luxury products, while another prefers budget items.
- **Why it matters:** Helps distinguish between distinct groups.

3. Compactness

- **Definition:** Points in a cluster should be close to the cluster center.
- **Example:** In K-means, compact clusters have low variance around centroids.
- **Why it matters:** Indicates tight grouping and reduces noise.

4. Separation

- **Definition:** Clusters should be well-separated from each other.
- **Example:** In image segmentation, pixels of different objects should fall into separate clusters.
- **Why it matters:** Prevents overlap and improves interpretability.

5. Scalability

- **Definition:** The clustering method should handle large datasets efficiently.
- **Example:** DBSCAN can scale well with spatial data but may struggle with high dimensions.
- **Why it matters:** Real-world data is often large and complex.

6. Stability

- **Definition:** Small changes in data should not drastically change clustering

results.

- **Example:** Adding a few new customers shouldn't reshuffle all existing clusters.
- **Why it matters:** Ensures reliability and robustness.

7. Discuss internal and external evaluation metrics used in clustering.

A. 1. Internal Evaluation Metrics

Definition:

Internal metrics evaluate the **quality of clusters** using **only the data** that was clustered — no external labels or ground truth are required.

They measure how **compact (cohesive)** and **well-separated** the clusters are.

Common Internal Metrics:

a) Silhouette Coefficient

- Measures how similar each point is to its own cluster (cohesion) compared to other clusters (separation).
- Ranges from **-1 to +1**.
 - **+1** → Well-clustered.
 - **0** → On the border between clusters.
 - **-1** → Wrongly assigned to a cluster.
- **Formula:**

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

where

- $a(i)$ = average distance from point i to all points in its own cluster
- $b(i)$ = lowest average distance from i to points in other clusters

Example:

If you cluster customers, a silhouette score of **0.8** means clusters are compact and well-separated.

b) Davies–Bouldin Index (DBI)

- Measures average similarity between each cluster and its most similar one.
- **Lower values** indicate **better clustering**.
- **Formula (simplified):**

$$DBI = \frac{1}{k} \sum_{i=1}^k \max_{j \neq i} \frac{S_i + S_j}{M_{ij}}$$

where S_i = intra-cluster distance and M_{ij} = inter-cluster distance.

c) Dunn Index

- Ratio of **minimum inter-cluster distance** to **maximum intra-cluster distance**.
- **Higher Dunn index** → better clustering.

2. External Evaluation Metrics

Definition:

External metrics compare the **obtained clusters** with a **known ground truth (actual labels)** to see how well the clustering matches the true class distribution.

Common External Metrics:**a) Rand Index (RI)**

- Measures the similarity between the predicted clusters and the true classes.
- Value ranges from **0 to 1**, where **1 means perfect match**.

b) Adjusted Rand Index (ARI)

- A corrected version of RI that adjusts for random chance.
- **Higher ARI** → better alignment with true labels.

c) Purity

- Measures how “pure” each cluster is — i.e., whether it mostly contains members from a single true class.
- **Formula:**

$$\text{Purity} = \frac{1}{N} \sum_k \max_j |C_k \cap T_j|$$

where C_k is cluster k and T_j is true class j .

- **Higher purity** → better clustering.

d) F-measure

- Combines **precision** and **recall** of clustering results compared to ground truth.
- **Higher F-measure** indicates better correspondence.

8. Explain the working of the K-means clustering algorithm with a diagram.**A. Step-by-Step Working of K-means**

1. **Initialization** Choose K initial centroids (either randomly or using methods like K-means++).
2. **Assignment Step** For each data point, compute its distance to each centroid and assign it to the **nearest cluster**.
 - Typically uses **Euclidean distance**:

$$d(x_i, \mu_k) = \sqrt{\sum_{j=1}^n (x_{ij} - \mu_{kj})^2}$$

3. **Update Step** Recalculate the centroid of each cluster as the **mean of all points** assigned to it:

$$\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$$

4. **Repeat** Alternate between assignment and update steps until:
 - Centroids no longer change (convergence), or

- A maximum number of iterations is reached.
5. **Output** Final cluster assignments and centroids.

```
import numpy as np  
import matplotlib.pyplot as plt  
from sklearn.cluster import KMeans
```

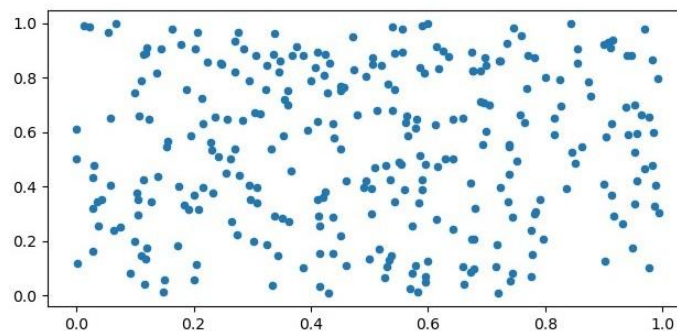
Step 2 – Generate Data

To test the K-Means algorithm, we need to generate some sample data. In this example, we will generate 300 random data points with two features. We will visualize the data also.

```
X = np.random.rand(300,2)
```

```
plt.figure(figsize=(7.5, 3.5))  
plt.scatter(X[:, 0], X[:, 1], s=20, cmap='summer');  
plt.show()
```

Output:



Step 3 – Initialize K-Means

Next, we need to initialize the K-Means algorithm by specifying the number of clusters (K) and the maximum number of iterations.

```
kmeans = KMeans(n_clusters=3, max_iter=100)
```

Step 4 – Train the Model

After initializing the K-Means algorithm, we can train the model by fitting the data to the algorithm.

```
kmeans.fit(X)
```

Step 5 – Visualize the Clusters

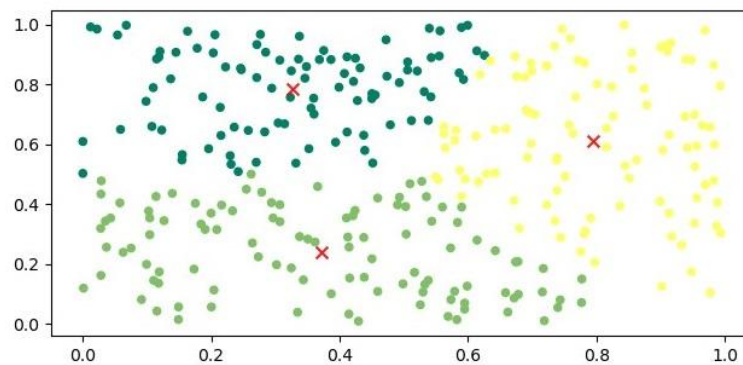
To visualize the clusters, we can plot the data points and color them based on their assigned cluster.

```
plt.figure(figsize=(7.5, 3.5))  
plt.scatter(X[:,0], X[:,1], c=kmeans.labels_, s=20, cmap='summer')  
plt.scatter(kmeans.cluster_centers_[0], kmeans.cluster_centers_[1],  
marker='x', c='r', s=50, alpha=0.9)  
plt.show()
```

Output:

The output of the above code will be a plot with the data points colored based

on their assigned cluster, and the centroids marked with an 'x' symbol in red color.



9. Compare supervised, unsupervised, and semi-supervised learning with examples.

A.

Aspect	Supervised Learning	Unsupervised Learning	Semi-Supervised Learning
Definition	Learns from labeled data	Learns from unlabeled data	Learns from a small amount of labeled data + large unlabeled data
Goal	Predict output labels	Discover hidden patterns	Improve prediction using limited labels
Data Requirement	Fully labeled dataset	No labels required	Partially labeled dataset
Algorithms	Linear regression, SVM, Decision Trees, Neural Networks	K-means, DBSCAN, PCA, Hierarchical clustering	Semi-supervised SVM, Label propagation, Self-training
Use Cases	Spam detection, disease diagnosis, sentiment analysis	Customer segmentation, anomaly detection, topic modeling	Web page classification, speech recognition, medical image analysis
Evaluation	Accuracy, precision, recall, F1-score	Internal/external clustering metrics	Combination of supervised metrics and clustering quality

10. Explain how the choice of distance metric can affect clustering results. Give examples with diagrams.

A. Clustering algorithms (like K-Means, Hierarchical, or DBSCAN) rely on distance measures to decide how close or far data points are from one another.

The **distance metric** determines:

- How clusters are **formed**,
- Their **shape**, and
- How data points are **assigned** to clusters.

Different distance measures can produce very different clusters even on the same dataset.

Common Distance Metrics:

Distance Metric	Formula	Best For
Euclidean Distance	$d(x, y) = \sqrt{\sum (x_i - y_i)^2}$	Continuous, numerical data; spherical clusters
Manhattan Distance	$d(x, y) = \sum x_i - y_i $	
Cosine Distance	$d(x, y) = 1 - \cos(\theta)$	
Minkowski Distance	Generalized form (depends on power p)	Flexible for mixed data
Mahalanobis Distance	Considers correlations between variables	Correlated or skewed data

Example 1: Euclidean vs. Manhattan Distance

Imagine three points in 2D:

A(1,1), B(4,1), C(1,4)

Metric	Distance between A & B	Distance between A & C
--------	------------------------	------------------------

Euclidean	3.0	3.0
-----------	-----	-----

Manhattan	3.0	3.0
-----------	-----	-----

Now consider D(3,3).

Using Euclidean distance, D is **closer to C**.

Using Manhattan distance, D may appear **closer to B** (depends on grid layout).

So, **cluster assignments can differ**.

Diagram 1 – Cluster Shape Difference

Euclidean Distance (Circular Clusters) Manhattan Distance (Square Clusters)



- Euclidean distance forms **circular/spherical clusters**.
- Manhattan distance forms **diamond/square-shaped clusters**.

Example 2: Cosine vs. Euclidean Distance

Used in **text or document clustering**.

- **Euclidean Distance:** Focuses on *magnitude* (length of vector).
- **Cosine Distance:** Focuses on *angle* (*direction*) between vectors.

Example:

Two text documents with the same word proportions but different lengths:

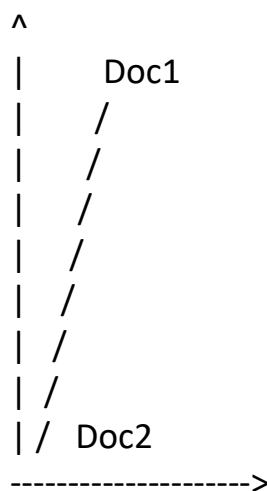
Doc1: "machine learning machine learning"

Doc2: "machine learning"

Euclidean distance sees them as **different** (because of length),

but Cosine distance sees them as **similar** (same direction in vector space).

Diagram 2 – Cosine vs. Euclidean



Same angle → High Cosine similarity

Different length → Large Euclidean distance

11. Why are similarity and distance measures crucial in clustering Illustrate with real-world data analysis examples.

A. Clustering algorithms aim to group data points such that those within the same cluster are more similar to each other than to those in other clusters. This similarity is quantified using **distance measures** (e.g., Euclidean, Manhattan, Cosine), which serve as the foundation for:

- **Determining cluster boundaries**
- **Assigning points to clusters**
- **Evaluating cluster compactness and separation**

Without appropriate distance metrics, clustering can produce misleading or meaningless groupings.

Real-World Examples

1. Customer Segmentation in Retail

- **Data:** Purchase history, frequency, spending amount
- **Distance Metric:** Euclidean or Manhattan
- **Use Case:** K-means clustering groups customers into segments like high-value, frequent buyers vs. occasional shoppers.
- **Impact:** Enables targeted marketing and personalized offers

2. Document Clustering for News Articles

- **Data:** TF-IDF vectors of article content
- **Distance Metric:** Cosine similarity
- **Use Case:** Hierarchical clustering groups articles by topic (e.g., politics, sports, tech).
- **Impact:** Improves search relevance and recommendation systems

3. Image Segmentation in Computer Vision

- **Data:** Pixel intensity, color histograms
- **Distance Metric:** Euclidean or Mahalanobis
- **Use Case:** Clustering pixels to identify objects or regions in an image.
- **Impact:** Used in medical imaging, autonomous vehicles, and facial recognition

4. Anomaly Detection in Network Security

- **Data:** Network traffic features (packet size, frequency)
- **Distance Metric:** Mahalanobis or Minkowski
- **Use Case:** DBSCAN or k-means isolates unusual traffic patterns.
- **Impact:** Detects potential intrusions or malware activity

5. Genomic Data Clustering

- **Data:** Gene expression levels
- **Distance Metric:** Pearson correlation or Euclidean
- **Use Case:** Clustering genes with similar expression profiles across conditions.
- **Impact:** Identifies gene functions and disease markers

Choosing the Right Metric

The effectiveness of clustering depends on matching the **distance measure to the data type**:

- **Numerical data** → Euclidean, Manhattan
- **Categorical data** → Hamming, Jaccard
- **Mixed-type data** → Gower's distance or kernel-based metrics

Poor metric choice can lead to:

- Overlapping clusters
- Misclassified points
- Reduced interpretability

12. Explain the spectral clustering algorithm in detail with steps and an example.

A. Step 1: Construct the Similarity Matrix

- Compute a similarity (affinity) matrix A where A_{ij} measures the similarity between data points x_i and x_j .
- Common choices:
 - **Gaussian (RBF) kernel:**

$$A_{ij} = \exp \left(-\frac{\|x_i - x_j\|^2}{2\sigma^2} \right)$$

- **k-nearest neighbors:** Connect each point to its k nearest neighbors.

Step 2: Build the Graph Laplacian

- Compute the **degree matrix** D , a diagonal matrix where $D_{ii} = \sum_j A_{ij}$.
- Compute the **unnormalized Laplacian**:

$$L = D - A$$

- Or use **normalized Laplacians**:
 - Symmetric: $L_{sym} = D^{-1/2} L D^{-1/2}$
 - Random walk: $L_{rw} = D^{-1} L$

Step 3: Compute Eigenvectors

- Compute the first k eigenvectors of the Laplacian matrix (smallest eigenvalues).
- Stack them column-wise to form a matrix $U \in \mathbb{R}^{n \times k}$.

Step 4: Cluster in the Embedded Space

- Treat each row of U as a point in $\mathbb{R}^k$.
- Apply **k-means** (or another clustering algorithm) to these points.

Step 5: Assign Original Points to Clusters

- The cluster assignment from the embedded space is mapped back to the original data points.

Example:

Credit Card Data Clustering Using Spectral Clustering

The below steps demonstrate how to implement Spectral Clustering using Sklearn. The data for the following steps is the **Credit Card Data** which can be downloaded from Kaggle

Step 1: Importing the required libraries

We will first import all the libraries that are needed for this project

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import SpectralClustering
from sklearn.preprocessing import StandardScaler, normalize
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
```

Step 2: Loading and Cleaning the Data

Changing the working location to the location of the data

```
cd "C:\Users\Dev\Desktop\Kaggle\Credit_Card"
```

Loading the data

```
X = pd.read_csv('CC_GENERAL.csv')
```

Dropping the CUST_ID column from the data

```
X = X.drop('CUST_ID', axis = 1)
```

Handling the missing values if any

```
X.fillna(method = 'ffill', inplace = True)
```

```
X.head()
```

Output:

	BALANCE	BALANCE_FREQUENCY	PURCHASES	ONEOFF_PURCHASES	INSTALLMENTS_PURCHASES	CASH_ADVANCE	PURCHASES_FREQUENCY	ONEC
0	40.900749	0.818182	95.40	0.00	95.4	0.000000	0.166667	
1	3202.467416	0.909091	0.00	0.00	0.0	6442.945483	0.000000	
2	2495.148862	1.000000	773.17	773.17	0.0	0.000000	1.000000	
3	1666.670542	0.636364	1499.00	1499.00	0.0	205.788017	0.083333	
4	817.714335	1.000000	16.00	16.00	0.0	0.000000	0.083333	

Step 3: Preprocessing the data to make the data visualizable

Preprocessing the data to make it visualizable

Scaling the Data

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

Normalizing the Data

```
X_normalized = normalize(X_scaled)
```

```
# Converting the numpy array into a pandas DataFrame
X_normalized = pd.DataFrame(X_normalized)
# Reducing the dimensions of the data
pca = PCA(n_components = 2)
X_principal = pca.fit_transform(X_normalized)
X_principal = pd.DataFrame(X_principal)
X_principal.columns = ['P1', 'P2']
X_principal.head()
```

	P1	P2
0	-0.489949	-0.679976
1	-0.519099	0.544828
2	0.330633	0.268879
3	-0.481656	-0.097612
4	-0.563512	-0.482505

Step 4: Building the Clustering models and Visualizing the Clustering

In the below steps, two different Spectral Clustering models with different values for the parameter 'affinity'. You can read about the documentation of the Spectral Clustering class [here](#). a) affinity = 'rbf'

```
# Building the clustering model
spectral_model_rbf = SpectralClustering(n_clusters = 2, affinity='rbf')

# Training the model and Storing the predicted cluster labels
labels_rbf = spectral_model_rbf.fit_predict(X_principal)

# Building the label to colour mapping
colours = {}
colours[0] = 'b'
colours[1] = 'y'

# Building the colour vector for each data point
cvec = [colours[label] for label in labels_rbf]

# Plotting the clustered scatter plot

b = plt.scatter(X_principal['P1'], X_principal['P2'], color='b');
y = plt.scatter(X_principal['P1'], X_principal['P2'], color='y');

plt.figure(figsize=(9, 9))
plt.scatter(X_principal['P1'], X_principal['P2'], c = cvec)
plt.legend((b, y), ('Label 0', 'Label 1'))
```

plt.show()
Output:

